Graph theory

Content:

1. Introduction
2. Common problems
3. Depth first search
4. Breadth first search
5. Topological sort
6. Shortest and longest path on a directed acyclic graph
7. Dijkstra's shortest path
8. Bellman Ford
9. Floyd Warshall all pairs shortest path
10. Bridges and articulation points
11. Tarjan's strongly connected
12. Traveling salesman
13. Eulerian paths and circuits
14. Prim's minimum spanning tree
15. Network flow:Max flow Ford Fulkerson
16. Network flow: Unweighted bipartite matching
17. Network flow: Mice and owls
18. Network flow: Edmonds Karp
19. Network flow: Capacity scaling
20. Network flow: Dinic's

1. Introduction

1. Storage, representation, and traversal
2. Point of view: computer science versus mathematics
    1. Algorithm implementation vs. proofs
    2. Graph is a mathematical term
        1. Network is the demotic

3. Canonical example: In a social network, how many friends does person X have?
    1. Also how many degrees of separation are between person X and person Y?
4. Characteristics
    1. Direction
        1. Undirected: edges have no orientation
            1. The edge (u, v) is identical to edge (v, u)
            2. An example is roads between cities
        2. Directed: edges have an orientation
            1. The edge from (u, v) is traversable, but the edge from (v, u) is not
            2. An example is people who bought each other gifts
    2. Weight
        1. Unweighted: edges have no weights
        2. Weighted: edges have weights that represent values
            1. The values could be cost, distance, or quantity
            2. Often the notation is included in the edge signature call
                1. E.g., (u, v, w), where w is the weight
    3. Special
        1. Trees
            1. A tree is an undirected, acyclic graph
        2. Rooted trees
            1. A rooted tree has a designated root node
            2. Every other node points either away from the root node or toward it
                1. Aborescence: edges point away from the root node
                1. The more common type
                2. Anti-aborescence: edges point toward the root node
        3. Directed acyclic graphs
            1. They often represent structures with dependencies
                1. Examples: scheduler, build system, compiler, university class prerequisites
                2. Common algorithms
                1. Find the shortest path

        2. Find a topological ordering of nodes
            1. For example: take biology and chemistry before genomics
    4. Bipartite graph
        1. Definitions
            1. Vertices can be split into two groups U and V
                1. Every edge connects between U and V
            2. The graph is two colorable
            3. There is no cycle with an odd length
        2. Common problem: what is the maximum matching possible?
            1. For example, between jobs and people
        3. Also, they are used in network flow
    5. Complete graphs
        1. There is a unique edge between every pair of nodes
        2. A complete graph with n vertices is denoted as K_n
        3. It is often considered the worst possible graph to encounter
            1. This is because of the large number of edges
            2. Therefore they are used to test algorithmic performance
5. Representation
    1. Data structures
        1. Adjacency matrix
            1. The cell m[i][j] represents the edge weight of going from node i to node j

	A	B	C	D
A	0	4	1	9
B	3	0	6	11
C	4	1	0	2
D	6	5	-4	0

            2. The cost of going from a node to itself is zero
                1. Therefore the diagonal of the matrix is all zeroes

2. Pros
    1. Space efficiency
    2. Constant edge weight lookup
    3. Simple implementation
3. Cons
    1. O (V ^ 2) space
        1. Graphs with 10,000 or more nodes become infeasible
    2. Iterating over all edges takes O (V ^ 2) time
        1. This is fine for dense graphs with lots of edges
        2. It is bad for sparse graphs
            1. Most cells are empty

2. Adjacency list
    1. A representation of a graph as a map from nodes to a list of edges
    2. Each node tracks all of its outgoing edges

A = [(B, 4), (C, 1)]

B = [(C, 6)]

C = [(A, 4), (B, 1), (D, 2)]

D = []

    1. The destination node and weight are included
    2. The source node is not, because it is the variable
3. Pros
    1. Good for sparse graphs
        1. It only tracks present edges
        2. Additional memory is not allocated for unused edges
    2. Efficiency for iteration over all edges
4. Cons
    1. Less space efficient on dense graphs
    2. It takes O (E) time to access the weight of a specific edge

        1. This rarely happens in practice
    3. Edge list
        1. An unordered list of edges

            [(C, A, 4), (A, C, 1), (B, C, 6), (A, B, 4), (C, B, 1), …]

        2. A simple representation
            1. But because there is no structure, therefore it is rarely used
        3. Pros
            1. Good for sparse graphs
            2. Easy to iterate over all edges
            3. Simple structure
        4. Cons
            1. Edge lookup is slow
            2. Memory issues with large graphs
2. **Common Problems**: often, new problems can be reduced to a familiar problem
    1. Shortest path problem
        1. Prompt: in a weighted graph, find the shortest path of edges between nodes A and B
        2. The graph could represent a road system
        3. Algorithmic solutions
            1. Breadth first search for unweighted graphs
            2. Dijkstra's
            3. Bellman-Ford
            4. Floyd-Warshall
            5. A*
    2. Connection
        1. Does there exist a path between node A and node B?
        2. Typical solutions
            1. Union find data structure

    2. A DFS or BFS
3. Negative cycles
    1. Negative cycle: a cycle whose total edge weight is negative
    2. Algorithmic solutions
        1. Bellman-Ford
        2. Floyd-Warshall
    3. Contexts in which negative edge weights are advantageous
        1. Finding arbitrage while trading currencies across an exchange
4. Strongly connected components in a directed graph
    1. Connected component: connected nodes in a disjunct graph
    2. SCC: every vertex in a cycle can reach every other vertex in the cycle
        1. It is not necessary that the reach happens in one edge
    3. Algorithmic solutions
        1. Tarjan's
        2. Kosaraju's
5. Traveling salesman
    1. Find the minimum cost path for traveling to each city and returning to the original city
    2. NP-hard: computationally challenging
    3. Algorithmic solutions
        1. Held-Karp with dynamic programming
        2. Branch and bound
        3. Approximation
            1. E.g., ant-colony optimization
6. Bridges
    1. Bridge: an edge that if removed thereby increases the number of connected components
    2. They often hint at weak points, bottlenecks, and vulnerabilities
    3. Examples: telephone network, a bridge of islands
7. Articulation points
    1. Articulation point: a node that if removed, increases the number of connected components
8. Minimum spanning tree (MST)

1. A subset of edges that connects all the vertices with a minimum total weight
    1. It is on an undirected, acyclic graph with weighted edges
    2. All nodes must be connected
2. Commonly contexts
    1. Least cost network
    2. Circuit
    3. Transportation network
3. Approximation algorithms that rely on MSTs
    1. Kruskal's
    2. Prim's & Boruvka's

9. Max network flow
    1. Flow network: edge weights represent capacity
        1. E.g., number of cars that fit on a road or volume through a pipe
    2. Common question
        1. With an infinite input source, how much flow can be pushed through the network?
        2. There is bound to be a bottleneck at some point in the network
        3. Once identified, the lower capacity edges can be improved

3. Depth first search

1. The most fundamental search algorithm for the exploration of nodes and edges of a graph
2. Runtime complexity is O (V + E), linear
3. Often it is used as a building block in other algorithms
    1. It is less useful by itself
4. Common uses
    1. Count connected components
        1. Algorithm
            1. Start a DFS at every node that has not been visited
            2. Mark all reachable nodes as being part of the same component
    2. Determine connectivity
    3. Find bridges and articulation points

    4. Compute a minimum spanning tree
    5. Detect and find cycles
    6. Check if a graph is bipartite
    7. Topological sort the nodes
    8. Find augmenting paths in a flow network
    9. Generate a maze
5. Course
    1. It indiscriminately searches the edges of the first node
        1. It backtracks once a node is found that has already been visited
        2. The DFS visits all the nodes on a branch
    2. Then it backtracks and finds another edge of the first node
        1. Then it searches all the nodes of the new edge

4. Breadth first search

1. Runtime complexity O (V + E)
2. Often it is used as a building block for other algorithms
3. Particular usefulness: find the shortest path on an unweighted graph
4. Course
    1. Start at a node
    2. Explore the neighbors of the first node before moving to the next level of neighbors
5. It uses a queue to remember which neighbors have been visited
6. Grid: a form of implicit graph
    1. The neighbors of a node can be determined by a location in the grid
    2. Often the grid is represented as an adjacency list or matrix
    3. Assume the grid is unweighted and that cells connect left, right, up, and down
    4. Direction vectors
        1. Often the transformation from grid to graph can be avoided
        2. Movement to the left, right, up, and down among cells is mathematical
            1. The row-column coordinate is (r, c)
            2. Add the vectors [-1, 0], [1, 0], [0, 1], and [0, -1]

(r – 1, c - 1)| (r – 1, c) | (r – 1, c + 1)

(r, c-1) | (r, c) | (r, c+ 1)

(r + 1, c - 1)| (r + 1, c) | (r + 1, c + 1)

3. This makes it easy to access neighboring cells from the current position
    1. Algorithm
        1. Loop over each direction vector
        2. Add it to the current position
        3. Skip invalid cells
            1. E.g., those that are out of bounds
5. A common problem: escape the dungeon
    1. Find the quickest way out of a 2D dungeon; some cells are not traversable

5. Topological sort

1. It is used to create a topological ordering on a directed graph
    1. Topological order
        1. For each edge from node A to node B, A appears before B in the order
2. Algorithm
    1. Pick an unvisited node
    2. Make a DFS
    3. On the recursive callback of the DFS, add the current node to the topological order
        1. Do the addition in reverse order
            1. i.e., from the end of an array to its beginning
3. Runtime complexity of O (V + E)
4. The topological order is not unique
    1. Multiple orders might exist for a graph
5. Not every graph has a topological order
    1. There is no possible topological order if there is a cyclic dependency
        1. There is nowhere to start

        2. Every node in the cycle depends on another
    2. Only DAGs can have a topological order
        1. To determine that a graph does not have a cycle, Tarjan's SCC algorithm often is used
    3. Every tree has a topological order
        1. Trees are acyclic
        2. Algorithm
            1. Select the leaf nodes
            2. Proceed by layers until the root
6. Real world situations for which topological sort is useful
    1. Class prerequisites
    2. Program dependencies
        1. Nodes represent programs
        2. Edges represent dependencies
    3. Event scheduling
    4. Assembly instructions

6. Shortest and longest path on a directed acyclic graph (DAG)

1. Runtime complexity of O (V + E)
    1. The next fastest algorithm for this problem is Dijkstra's
        1. But it works only on non-negatively weighted graphs
        2. With SSP on a DAG, weights can be positive or negative
2. Algorithm
    1. Find a topological order
    2. Process each node sequentially
        1. Relax each edge as it is seen
        2. Relax an edge: update the edge weight only if a shorter path can be obtained
3. Finding the longest path by edge weight on a DAG
    1. On a general graph, this is NP-hard
    2. On a DAG, the runtime complexity is O (V + E)
    3. Algorithm

1. Multiply each edge value by -1
2. Find the shortest path
3. Again multiply the edge values by -1

7. Dijkstra's shortest path

1. A single source shortest path (SSSP) algorithm for graphs with non-negative edge weights
    1. The constraint of non-negativity
        1. This ensures that once a node is visited, its distance is optimal
    2. An arbitrary starting point must be specified
    3. The algorithm returns the shortest path between the start and all other nodes
    4. Time complexity is O(E * log(V))
2. Algorithm
    1. Maintain a distance array
        1. The distance to every node is positive infinity
        2. The distance to the start node is zero
    2. Maintain a priority queue (PQ) of key-value pairs
        1. The pair denotes node index and distance
        2. They instruct on the next node to visit based on a sorted minimum value
    3. Insert (s, 0) into the PQ
        1. Loop while PQ is not empty
            1. Pull out the next most promising key-value pair
    4. Iterate over all the edges outward from the current node
        1. Relax each edge
            1. Append a new key-value pair to the PQ for every relaxation
3. Optimizations
    1. Stopping early
        1. If the destination has been visited, its shortest distance will not be changed
    2. Eager Dijkstra's uses an indexed PQ to update values
        1. The lazy method inserts duplicate key-value pairs
        2. I t is more efficient to insert a duplicate than to update

        1. O(log n) vs. O(n)
    2. In a dense graph, this approach is inefficient
        1. Several outdated key-value pairs accumulate in the PQ
    3. The eager method avoids duplicate key-value pairs
        1. The indexed PQ makes the updates in O(log n)
3. D-ary heap
    1. D-ary heap: each node has at most d children
        1. The structure increases the speed of decrease key operations (i.e., updates)
        2. It decreases the speed of removals
    2. In dense graphs, there are more updates than dequeue operations
    3. The optimal d-ary heap degree formula is D = E / V
        1. The time complexity is $O(E * \log_{e/v} (V))$
4. The state of the art
    1. The Fibonacci heap gives Dijkstra's a time complexity of O(E + Vlog(V))
    2. In practice, Fibonacci heaps are difficult to implement
    3. Also, they have a large constant amortized overhead
        1. They are impractical unless the graph is large

8. Bellman Ford

1. A SSSP algorithm, but with a slow time complexity of O(EV)
    1. It is better to use Dijkstra's
    2. Dijkstra's is not possible with negative edge weights
        1. Then Bellman Ford becomes useful
2. Algorithm
    1. Define variables
        1. E for the number of edges
        2. V for the number of vertices
        3. S for the id of the starting node
        4. D for an array the size of V
            1. It tracks the best distance from S to each node

2. Set every entry in D to positive infinity
3. Set D[S] to zero
4. Relax each edge V – 1 times
    1. The edges do not need to be processed in any specific order
5. To detect negative cycles, run the algorithm a second time
    1. Check for nodes that update to a better value than the known best value
    2. This change indicates a negative cycle
        1. Mark the node with a cost of negative infinity

9. Floyd Warshall all pairs shortest path (APSP)

1. It can find the shortest path between all pairs of nodes
2. The time complexity is O(V^3)
    1. It is ideal for graphs with no more than a few hundred nodes
    2. It can detect negative cycles
3. The optimal representation is a two dimensional adjacency matrix
    1. If there is no value between nodes, set its value to positive infinity
4. The main idea
    1. Gradually build all intermediate routes between nodes i an d j
5. Create a dynamic programming (DP) memo table
    1. The table caches previous optimal solutions
    2. Implement it as a 3D matrix of size n x n x n
    3. An example index is dp[k][i][j]
        1. It represents the shortest path from i to j
        2. It routes through nodes {0, 1, … , k – 1, k}
    4. The table uses O(V^3) memory
        1. An optimization is to compute the solution for k in place
            1. This results in a space complexity of O(V^2)

10. Bridges and articulation points

1. An analogy: if connected components are islands, a bridge connects them

2. Bridge algorithm
    1. DFS from any node
        1. Label nodes with an increasing value id
        2. Keep track of the id of each node
        3. Keep track of the smallest low-link value
            1. The smallest id reachable from the current node
            2. This includes itself
            3. Often nodes with the same low-link value belong to the same SCC
                1. This depends on the location of the start of the search
                2. The low-link values might be incorrect
        4. Identification of bridges
            1. They occur where the id of the node is less than the low link value of the destination
    2. Complete V more DFS to label all the nodes
3. The time complexity of the bridge algorithm is O(V(V+E))
    1. An optimization is to update the low-link values in one pass
        1. This results in O(V + E) time complexity
4. Articulation points
    1. An observation
        1. Assume a connected component with three or more vertices
            1. If any edge (u, v) is a bridge, then either u or v is an articulation point
        2. There are cases in which there is a bridge but no articulation point
            1. The presence of a cycle implies that the node is an articulation point
            2. A cycle corresponds with a SCC
            3. An exception to the cyclical indication
                1. If the starting node
                1. Has no outgoing edges
                2. Has only one outgoing edge
                2. Thus, to be an articulation point, there must be more than one outgoing edge
    2. Algorithm
        1. A modification of the bridge algorithm

2. Keep track of the outgoing edges

11. Tarjan's strongly connected components

1. The algorithm uses a stack invariant
    1. It prevents SCCs from interfering with the low-link values of other SCCs
    2. The stack invariant
        1. Valid nodes from which to update low-link values
        2. Nodes are aded to the stack as they are first explored
        3. They are removed each time a complete SCC is found
    3. New low-link update condition
        1. If there are nodes u and v
            1. To update the low-link value of node u with the low-link value of node v
                1. There has to be a path of edges from u to v
                2. Node v must be on the stack
    4. Also low-link values are found during the DFS
        1. This gives O(V + E) time complexity
2. Algorithm
    1. Mark the id of each node as unvisited
    2. Start the DFS
        1. While visiting each node
            1. Assign the node an id and a low-link value
            2. Mark the current nodes as visited
            3. Add the current nodes to a seen stack
    3. On the DFS callback
        1. If the previous node is on the stack
            1. Update the min of the low-link value of the current node
                1. It is updated with the low-link value of the last node
            2. This allows low-link values to propagate throughout cycles
    4. After visiting all neighbors
        1. If the current node started a connected component

1. Pop nodes of the stack until the current node is reached
2. A node started a connected component if its id equals its low-link value

12. Traveling salesman problem with dynamic programming

1. Prompt: find the minimum cost Hamiltonian cycle in a complete, weighted graph
2. Representation: adjacency matrix
3. Often there are multiple valid solutions
4. The problem is NP-complete
    1. It is difficult to find an optimal solution for large inputs
5. Brute force solution
    1. O(n!) time complexity
    2. Algorithm
        1. Compute the cost of every possible tour
    3. It is optimal if the graph has six or fewer nodes
6. Optimization: dynamic programming
    1. O(n^2 * 2^n)
    2. Main idea
        1. Compute the optimal solution for all the subpaths of length N
        2. Do this while using information from the already known optimal partial tours
            1. Those tours have a length of N – 1
    3. Set up
        1. Selection of the starting node
            1. Any node whose index is greater than or equal to zero and smaller than N
        2. Then compute and store the optimal value from S to each node X
            1. This solves the TSP for all paths of length n = 2
    4. Dynamic programming state: it contains two values
        1. Values
            1. The set of visited nodes in the subpath
            2. The index of the last visited node in the path
        2. The dynamic programming state requires O(N2^2) space

        1. The best way to achieve this is to use a single 32-bit integer
            1. It is compact, quick, and allows for easy caching in a memo table
    5. To complete the tour, connect the tour back to the start node S
        1. Loop over the end state in the memo table for every possible end position
            1. Minimize the lookup value plus the cost of going back to S

13. Existence of Eulerian paths and circuits

1. Eulerian path: a path of edges that visits all the edges in a graph exactly once
    1. Not every graph has an Eulerian path
    2. There are a subset of valid starting nodes for a graph that has an Eulerian path
    3. Detection
        1. At least one vertex has an out degree minus in degree equal to one
        2. At least one vertex has an in degree minus out degree equal to one
        3. All other vertices have equal in and out degrees
2. Eulerian circuit: an Eulerian path that starts and ends on the same vertex
    1. An Eulerian circuit on an undirected graph
        1. Every vertex has an even degree
    2. On a directed graph
        1. Every vertex has an equal in degree and out degree
3. For Eulerian paths and circuits, and addition constraint
    1. All vertices with a non-zero degree must belong to the same connected component
4. Nodal degree
    1. In an undirected graph, the degree is the number of edges
    2. In a directed graph, there are two types of degrees
        1. In degrees
        2. Out degrees

14. Eulerian path algorithm

1. Algorithm
    1. Loop through all the edges to count the in and out degrees of each node

        1. Keep two arrays, one for in and the other for out degrees
    2. Find a valid starting node
        1. A valid starting node has only one outgoing edge
        2. A valid end node has only one incoming edge
        3. Note
            1. If all in and out degrees are equal, then it is an Eulerian circuit
            2. Any node with a non zero degree can be the starting node
    3. Modify a DFS to visit all the edges of the graph
        1. Take edges on the recursive callback until no unvisited edges remain
        2. Every time an edge is taken, reduce the outgoing edge count in the array
            1. This indicates when a node has no more edges

2. The time complexity is O(E)

15. Eager Prim's minimum spanning tree

1. A greedy MST that works well on dense graphs
2. It is faster than its popular rivals: Kruskal's and Boruvka's
3. Prim's has difficulty finding the minimum spanning forest on a disconnected graph
    1. The algorithm must be run on each connected component individually
4. Runtime complexity
    1. The lazy version of Prim's has a runtime of O(E * log E)
    2. The eager version has a runtime of O(E * log V)
5. Lazy Prim's algorithm
    1. Maintain a minimum PQ
        1. It sorts edges based on min cost
        2. It is used to determine the next node to visit and the edge to traverse
    2. Start on any node
        1. Mark it as visited
        2. Iterate over all the edges of S
            1. Add them to PQ
    3. While PQ is not empty and the MST has not been formed

    1. Dequeue the next cheapest edge from the PQ
    2. If the dequeued edge is outdated, skip it and poll again
        1. An outdated edge has already been visited
        2. To poll is to dequeue
    3. Mark the current node as visited and add the selected edge to the MST
4. Iterate over the edges of the new current node
    1. Add all its edges to the PQ
    2. Do not add edges to the PQ that point to already visited nodes

6. Key realization
    1. For any MST with directed edges
        1. Each node is paired with exactly one of its incoming edges
        2. An exception is the start node
7. Eager Prim's
    1. Determine which incoming edge of a node should be included in the MST
    2. A difference between the eager and lazy methods
        1. Lazy: add edges to the PQ during iteration over the edges of a node
        2. Eager: relax the most promising edge of the destination node
    3. Efficient retrieval and update of the (node, edge) pairs
        1. Possibilities
            1. Indexed priority queue (IPQ)
                1. Time complexity of O(E * log V)
    4. Algorithm
        1. Maintain a minimum IPQ of size V
            1. It sorts vertex-edge pairs (v, e) based on the minimum edge cost of e
        2. Start on any node S
            1. Mark s as visited
            2. Relax all the edges of S
        3. While the IPQ is not empty and the MST has not been formed
            1. Dequeue the next best (v, e) pair from the IPQ
            2. Mark the node v as visited

        3. Add the edge e to the MST
    4. Relax all edges of v
        1. Do not relax any edge pointing to an already visited node
        2. Relax: update the entry if the new edge has a lower cost than the old edge

16. Network flow: Ford Fulkerson

1. Flow graph
    1. Directed edges
        1. Also called arcs
    2. Characteristics of edges
        1. Maximum capacity
            1. At all times the flow must be less than or equal to the maximum capacity
        2. Flow value: the number of units of flow that pass through the edge
            1. Initially it is zero for all edges, until the max flow algorithm is run
    3. Types of nodes
        1. Source (s)
        2. Sink (t)
            1. Derivation of the maximum flow
                1. Achieved by the sum of the edges attached to the sink node
    4. There might be multiple ways of achieving maximum flow
        1. For example, if each edge is given different flow values
        2. Different ways nonetheless have the same overall maximum flow value
2. Ford Fulkerson method
    1. Derivation of the maximum flow
        1. Repeated finding of the augmenting paths through the residual graphs
        2. Augmentation of the flow until no more augmenting paths are found
    2. Augmenting path: a path of edges in the residual graph with unused capacity
        1. The unused capacity must be greater than zero from the source to the sink
        2. It can flow only through non saturated edges
        3. An indication of the achievement of maximum flow is the absence of augmenting paths

    4. Ford Fulkerson does not specify how to find augmenting paths: for flexibility
        1. A DFS can find augmenting paths
    5. Every augmenting path has a bottleneck value
        1. It is the smallest edge along the path
        2. It can be found by taking the capacity and current flow of an edge
    6. The bottleneck value can be used to augment the path flow
    7. Augmenting the flow: updating flow values of the edges along the augmenting path
        1. For forward edges, this increases the flow by the bottleneck value
        2. For back or residual edges, this decreases the flow by the bottleneck value
    8. Residual edges undo bad augmenting paths that do not lead to maximum flow
        1. In later iterations, they become valid paths
3. Residual graph: the graph that contains residual edges
    1. Often this is referred to as the flow graph
4. Remaining capacity of an edge e
    1. The difference the capacity and flow of e
        1. It ensures that the remaining capacity of an edge is always non negative
        2. The flow can be non negative

3. Algorithm
    1. Until no more augmenting paths from s to t exist
        1. Find augmenting paths
        2. Augment the flow
    2. Key realization
        1. The sum of the bottlenecks found in each augmenting paths equals the max flow
4. Time complexity
    1. Determined by the algorithm used to find augmenting paths
    2. With a DFS, that is O(f * E), where f is maximum flow
    3. Other, faster algorithms
        1. Edmonds-Karp: BFS to find augmenting paths has O(E^2 * V)
        2. Capacity scaling: a heuristic to pick larger paths first has O(E^2 * log U)
        3. Dinic's: BFS and DFS to find augmenting paths has O(V^2 * E)

    4. Push relabel: maintains a pre-flow for max flow and has O(V^2 * E)
  4. Note that computational complexity and practical operations differ
    1. Many of the computations are pessimistic; the practical algorithms run faster

17. Network flow: unweighted bipartite matching

1. Maximum cardinality matching (MCBM): a maximization of the pairs that can be matched
  1. A greedy approach does not work
  2. A better approach is to find the max flow
    1. Make the edges directed
    2. Add unit capacities to each edge
      1. 0/1 indicates 0 flow and a maximum capacity of 1
        1. The maximum capacity of 1 ensures only one edge traversal from the node
        2. With a larger maximum capacity, more edges are traversable from each node
    3. Introduce two new nodes: source and sink
      1. Connect the source one of the columns of nodes
      2. Connect the second column of nodes to the sink
    4. Use a maximum flow algorithm to push flow through the network
      1. This shows what edges are populated with flow
      2. With that information, the maximum matching can be reconstructed

18. Network flow: mice and owls

1. Prompt
  1. Constraints
    1. An owl looks upon a field of mice
    2. There are holes next to the mice with a space capacity
    3. Each mouse has a limited running distance before it is caught by the owl
  2. Question
    1. What is the maximum number of mice that can hide safely?
2. Key realization: the graph is bipartite
  1. Set up a flow graph and run a maximum flow algorithm

    2. The highest number of safe mice equals the value of the maximum flow
3. Algorithm
    1. Figure out which holes each mouse can reach
    2. Match mice to holes to maximize the safety of the group

19. Network flow: elementary math

1. Prompt
    1. Which three operations (+, -, x) should correspond with a pair of numbers on a math test?
2. Key realization: for every pair, at most three unique solutions are produced
    1. This allows the formulation of the flow graph as bipartite
        1. Input pairs are on one side
        2. Solutions are on the other side
    2. Then assign capacity from the source and to the sink, each with a capacity of 1
    3. Run the maximum flow algorithm
    4. Look at the matching
        1. Figure out which matchings were used

20. Network flow: Edmonds Karp

1. Prompt: find the maximum flow on a flow graph
2. A method of augmentation
    1. It repeatedly finds the shortest path from s to t
        1. It does so in terms of the number of edges used in each iteration
3. Strategy: optimize the DFS of the Ford Fulkerson algorithm
    1. Instead of using a DFS to find augmenting paths, use a BFS
        1. The new time complexity is O(V * E^2)
        2. This is advantageous
            1. The time complexity does not depend on the capacity value of any edge
                1. It is independent of the maximum flow
            2. Note: strongly polynomial algorithms do not depend directly on input values
    2. The edges are treated as unweighted

        1. Here unweighted indicates that all positive capacity are evaluated the same
    3. DFS can lead to zigzagging through the flow graph
        1. Zigzagging can find longer augmenting paths
        2. Longer augmenting paths increase the chance for a small bottleneck value
        3. A small bottleneck value results in a longer runtime
4. Algorithm
    1. Make a BFS from the source to sink
        1. Only edges with a capacity greater than zero are traversable
        2. Find paths until the graph is saturated
            1. i.e., there are no more augmenting paths
    2. The maximum flow from Edmonds Karp is the sum of the bottleneck values
        1. Similarly, the values of the capacities that enter the sink can be summed

21. Network flow: capacity scaling

1. A heuristic
    1. Prioritize the traversal of edges with larger capacities
        1. This avoids paths that have a small bottleneck
        2. A technical statement
            1. Traverse only edges whose remaining capacity is greater than or equal to delta
            2. Delta functions as a threshold for the acceptance of traversable edges
2. Algorithm
    1. Variable definitions
        1. U: the value of the largest edge capacity in the initial flow graph
        2. Delta: the largest power of two less than or equal to U
    2. Repeatedly find augmenting paths through the flow graph
        1. The paths must have remaining capacity greater than or equal to delta
    3. Once no more paths satisfy the criteria
        1. Decrease the value of delta by dividing it by two
        2. Repeat this step while delta is larger than zero
3. Time complexity: O(E^2 * log U) or O(E * V * log U)

22. Network flow: Dinic's algorithm

1. A fast, strongly polynomial maximum flow algorithm
2. Runtime complexity: O(V^2 * E)
    1. On bipartite graphs: O(the square of V * E)
        1. It achieves this by reducing to Hopcroft-Karp
3. Main idea: guide augmenting paths from the sink to source by a level graph
    1. The levels of the graph are obtained by a BFS from the source
    2. An edge is part of the level graph only if it makes progress toward the sink
        1. i.e., it moves from level L to level L + 1
    3. Residual edges can be made part of the level graph
        1. They must have a remaining capacity greater than zero
4. Algorithm
    1. Construct a level graph
        1. Traverse with a BFS from the source
            1. Label all the levels of the current flow graph
    2. If the sink was never reached, stop and return the maximum flow
    3. Traverse with multiple DFSs from s to t
        1. Use only valid edges in the level graph
        2. Traverse until a blocking flow is reached
        3. Sum over the bottleneck values of all augmenting paths
            1. This calculates the maximum flow
    4. Repeat steps one through three
5. Optimizations
    1. Multiple dead ends can be found
        1. The same dead end can be reached during the DFSs
    2. Optimization
        1. Remove the dead ends of the level graph before each blocking flow phase